

Technical Implementation Guide: Dual Authentication & SSO

This document contains the complete, end-to-end code and configuration mapping for the dual-authentication integration (Azure Entra ID SSO & Local JWT) across the UI and API applications.

1. Global Configuration

appsettings.json (Shared Structure)

```
{
  "KeyVault": {
    "VaultUri": "https://kv-intervention-dev.vault.azure.net/"
  },
  "AuthenticationFeatures": {
    "EnableSSO": true,
    "EnableLocalLogin": true
  },
  "LocalJwt": {
    "Issuer": "InterventionManagement.Local",
    "Audience": "InterventionManagement.Api",
    "TokenLifetimeMinutes": 60
  },
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "YOUR_TENANT_ID",
    "ClientId": "YOUR_CLIENT_ID",
    "Audience": "api://intervention-management-api",
    "CallbackPath": "/signin-oidc"
  },
  "Graph": {
    "BaseUrl": "https://graph.microsoft.com/v1.0",
    "Scopes": "Mail.Send User.Read"
  },
  "DownstreamApi": {
    "BaseUrl": "https://localhost:7036/",
    "Scopes": [ "api://intervention-management-api/access_as_user" ]
  },
  "Swagger": {
    "OAuthClientId": "YOUR_SWAGGER_CLIENT_ID",
    "OAuthScopes": {
      "api://intervention-management-api/access_as_user": "Access Intervention Management API"
    }
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

2. UI Application Implementation

Program.cs

```
using Azure.Extensions.AspNetCore.Configuration.Secrets;
using Azure.Identity;
using InterventionManagement.Ui.Infrastructure;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Authentication.OpenIdConnect;
using Microsoft.AspNetCore.HttpOverrides;
using Microsoft.Identity.Web;
using Microsoft.Identity.Web.UI;

var builder = WebApplication.CreateBuilder(args);
builder.Configuration.AddJsonFile("appsettings.local.json", optional: true, reloadOnChange: true);

var vaultUri = builder.Configuration["KeyVault:VaultUri"];
if (!string.IsNullOrEmpty(vaultUri))
{
    builder.Configuration.AddAzureKeyVault(new Uri(vaultUri), new DefaultAzureCredential(), new
    AzureKeyVaultConfigurationOptions());
}

var scopesSection = builder.Configuration.GetSection("DownstreamApi:Scopes");
var apiScopes = scopesSection.Get<string[]>() ?? new[] { scopesSection.Value ?? string.Empty }.Where(s
=> !string.IsNullOrEmpty(s)).ToArray();

var enableSso = builder.Configuration.GetValue("AuthenticationFeatures:EnableSSO", false);

var authBuilder = builder.Services.AddAuthentication(options =>
{
    options.DefaultScheme = CookieAuthenticationDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = CookieAuthenticationDefaults.AuthenticationScheme;
})
.AddCookie(CookieAuthenticationDefaults.AuthenticationScheme, options =>
{
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/Login";
});

if (enableSso)
{
    authBuilder.AddMicrosoftIdentityWebApp(builder.Configuration.GetSection("AzureAd"))
        .EnableTokenAcquisitionToCallDownstreamApi(apiScopes)
        .AddMicrosoftGraph(builder.Configuration.GetSection("Graph"))
        .AddDistributedTokenCaches();
}

builder.Services.AddAuthorization();
builder.Services.AddInfrastructure(builder.Configuration, enableSso);
builder.Services.AddControllersWithViews().AddMicrosoftIdentityUI();

var app = builder.Build();

app.UseForwardedHeaders(new ForwardedHeadersOptions
{
    ForwardedHeaders = ForwardedHeaders.XForwardedProto | ForwardedHeaders.XForwardedHost
});
```

```

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Dashboard}/{action=Index}/{id?}");

app.Run();

```

Infrastructure/DependencyInjection.cs

```

using InterventionManagement.Ui.Application.Abstractions;
using InterventionManagement.Ui.Infrastructure.Api;
using InterventionManagement.Ui.Infrastructure.Graph;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace InterventionManagement.Ui.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(this IServiceCollection services,
        IConfiguration configuration, bool enableSso)
    {
        services.AddHttpContextAccessor();
        services.AddTransient<TokenAuthorizationHandler>();

        if (enableSso)
        {
            services.AddScoped<IUserProfileService, GraphUserProfileService>();
        }
        else
        {
            services.AddScoped<IUserProfileService, LocalUserProfileService>();
        }

        services.AddHttpClient<IIInterventionApiClient, SecureInterventionApiClient>(client =>
        {
            client.BaseAddress = new Uri(configuration["DownstreamApi:BaseUrl"]
                ?? throw new InvalidOperationException("DownstreamApi:BaseUrl is missing."));
        })
        .AddHttpMessageHandler<TokenAuthorizationHandler>();

        return services;
    }
}

```

Infrastructure/Api/TokenAuthorizationHandler.cs

```
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Identity.Web;
using System.Net.Http.Headers;

namespace InterventionManagement.Ui.Infrastructure.Api;

public class TokenAuthorizationHandler(
    IHttpContextAccessor httpContextAccessor,
    IConfiguration configuration) : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(HttpRequestMessage request,
        CancellationToken cancellationToken)
    {
        var context = httpContextAccessor.HttpContext;

        if (context?.User.Identity?.IsAuthenticated == true)
        {
            string? token = null;
            var isLocalLogin = context.User.FindFirst("idp")?.Value == "local";

            if (isLocalLogin)
            {
                token = await context.GetTokenAsync("access_token");
            }
            else
            {
                var tokenAcquisition = context.RequestServices.GetService<ITokenAcquisition>();
                if (tokenAcquisition != null)
                {
                    var scopesSection = configuration.GetSection("DownstreamApi:Scopes");
                    var scopes = scopesSection.Get<string[]>() ?? new[] { scopesSection.Value ??
string.Empty }.Where(s => !string.IsNullOrEmpty(s)).ToArray();
                    token = await tokenAcquisition.GetAccessTokenForUserAsync(scopes);
                }
            }

            if (!string.IsNullOrEmpty(token))
            {
                request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token);
            }
        }

        return await base.SendAsync(request, cancellationToken);
    }
}
```

Infrastructure/Api/SecureInterventionApiClient.cs

```
using System.Net.Http.Json;
using InterventionManagement.Ui.Application.Abstractions;
using InterventionManagement.Ui.Application.DTOs;
```

```

namespace InterventionManagement.Ui.Infrastructure.Api;

public sealed class SecureInterventionApiClient(HttpClient client) : IInterventionApiClient
{
    public Task<IReadOnlyList<InterventionDto>> GetDashboardAsync(CancellationTokens cancellationTokens)
    =>
        GetAsync<IReadOnlyList<InterventionDto>>("api/reports/dashboard", cancellationTokens);

    public Task<IReadOnlyList<CalendarEventDto>> GetCalendarAsync(CancellationTokens cancellationTokens)
    =>
        GetAsync<IReadOnlyList<CalendarEventDto>>("api/reports/calendar", cancellationTokens);

    public Task<InterventionDto?> GetAsync(Guid id, CancellationTokens cancellationTokens) =>
        GetNullableAsync<InterventionDto>($"api/interventions/{id}", cancellationTokens);

    public async Task CreateAsync(UpsertInterventionRequest request, CancellationTokens cancellationTokens)
    {
        using var response = await client.PostAsJsonAsync("api/interventions", request,
cancellationTokens);
        response.EnsureSuccessStatusCode();
    }

    public async Task UpdateAsync(Guid id, UpsertInterventionRequest request, CancellationTokens cancellationTokens)
    {
        using var response = await client.PutAsJsonAsync($"api/interventions/{id}", request,
cancellationTokens);
        response.EnsureSuccessStatusCode();
    }

    public async Task ApplyActionAsync(Guid id, string action, CancellationTokens cancellationTokens)
    {
        using var response = await client.PostAsJsonAsync($"api/interventions/{id}/actions", new
InterventionActionRequest(action), cancellationTokens);
        response.EnsureSuccessStatusCode();
    }

    public Task<EmailNotificationSettingsDto> GetEmailSettingsAsync(CancellationTokens cancellationTokens) =>
        GetAsync<EmailNotificationSettingsDto>("api/settings/email-notifications", cancellationTokens);

    public async Task SaveEmailSettingsAsync(EmailNotificationSettingsDto request, CancellationTokens cancellationTokens)
    {
        using var response = await client.PostAsJsonAsync("api/settings/email-notifications", request,
cancellationTokens);
        response.EnsureSuccessStatusCode();
    }

    public Task<UserAuthorizationDto> GetCurrentAuthorizationAsync(CancellationTokens cancellationTokens) =>
        GetAsync<UserAuthorizationDto>("api/me/authorization", cancellationTokens);

    private async Task<T> GetAsync<T>(string path, CancellationTokens cancellationTokens)
    {
        return await client.GetFromJsonAsync<T>(path, cancellationTokens)
            ?? throw new InvalidOperationException($"API returned no payload for {path}.");
    }

    private async Task<T?> GetNullableAsync<T>(string path, CancellationTokens cancellationTokens)
    {

```

```

        using var response = await client.GetAsync(path, cancellationToken);
        if (response.StatusCode == System.Net.HttpStatusCode.NotFound)
        {
            return default;
        }

        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<T>(cancellationToken);
    }
}

```

Controllers/AccountController.cs

```

using System.Security.Claims;
using System.Net.Http.Json;
using InterventionManagement.Ui.Application.DTOS;
using InterventionManagement.Ui.Web.Models;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Authentication.OpenIdConnect;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace InterventionManagement.Ui.Web.Controllers;

[AllowAnonymous]
public sealed class AccountController(IConfiguration configuration, IHttpClientFactory
httpClientFactory) : Controller
{
    [HttpGet]
    public IActionResult Login(string? returnUrl = null, string? error = null) =>
        View(new LoginViewModel
        {
            EnableSSO = configuration.GetValue("AuthenticationFeatures:EnableSSO", true),
            EnableLocalLogin = configuration.GetValue("AuthenticationFeatures:EnableLocalLogin",
false),
            ReturnUrl = returnUrl,
            ErrorMessage = error
        });

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult ExternalLogin(string? returnUrl = null)
    {
        if (!configuration.GetValue("AuthenticationFeatures:EnableSSO", true))
        {
            return RedirectToAction(nameof(Login), new { error = "Single sign-on is disabled.",
returnUrl });
        }

        return Challenge(new AuthenticationProperties { RedirectUri = returnUrl ?? Url.Action("Index",
"Dashboards") }, OpenIdConnectDefaults.AuthenticationScheme);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> LocalLogin(string username, string password, string? returnUrl =
null, CancellationToken cancellationToken = default)
    {

```

```

        if (!configuration.GetValue("AuthenticationFeatures:EnableLocalLogin", false))
        {
            return RedirectToAction(nameof(Login), new { error = "Local login is disabled.",
returnUrl });
        }

        if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
        {
            return View(nameof(Login), new LoginViewModel
            {
                EnableSSO = configuration.GetValue("AuthenticationFeatures:EnableSSO", true),
                EnableLocalLogin = true,
                returnUrl = returnUrl,
                ErrorMessage = "Invalid local credentials.",
                Username = username
            });
        }

        var apiToken = await GetLocalApiTokenAsync(username, password, cancellation_token);
        if (apiToken is null)
        {
            return View(nameof(Login), new LoginViewModel
            {
                EnableSSO = configuration.GetValue("AuthenticationFeatures:EnableSSO", true),
                EnableLocalLogin = true,
                returnUrl = returnUrl,
                ErrorMessage = "Unable to obtain a local API token.",
                Username = username
            });
        }

        var claims = new[]
        {
            new Claim(ClaimTypes.Name, username),
            new Claim("oid", "00000000-0000-0000-0000-000000000111"),
            new Claim("idp", "local")
        };

        var authProperties = new AuthenticationProperties();
        authProperties.StoreTokens(new[]
        {
            new AuthenticationToken { Name = "access_token", Value = apiToken.AccessToken }
        });

        await HttpContext.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationScheme,
            new ClaimsPrincipal(new ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationScheme)),
            authProperties);

        return LocalRedirect(returnUrl ?? Url.Action("Index", "Dashboard") ?? "/");
    }

    [Authorize]
    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Logout()
    {
        var callbackUrl = Url.Action("Login", "Account", values: null, protocol: Request.Scheme);
        var isLocalLogin = User.FindFirstValue("idp") == "local";

        if (configuration.GetValue("AuthenticationFeatures:EnableSSO", false) && !isLocalLogin)

```

```

        {
            return SignOut(new AuthenticationProperties { RedirectUri = callbackUrl },
                CookieAuthenticationDefaults.AuthenticationScheme,
                OpenIdConnectDefaults.AuthenticationScheme);
        }

        return SignOut(new AuthenticationProperties { RedirectUri = callbackUrl },
            CookieAuthenticationDefaults.AuthenticationScheme);
    }

    private async Task<LocalLoginResponse?> GetLocalApiTokenAsync(string username, string password,
        CancellationToken cancellationToken)
    {
        var client = httpClientFactory.CreateClient();
        client.BaseAddress = new Uri(configuration["DownstreamApi:BaseUrl"]!);
        using var response = await client.PostAsJsonAsync("api/auth/local-login", new
        LocalLoginRequest(username, password), cancellationToken);
        if (!response.IsSuccessStatusCode)
        {
            return null;
        }

        return await response.Content.ReadFromJsonAsync<LocalLoginResponse>(cancellationToken);
    }
}

```

3. API Application Implementation

Program.cs

```

using Azure.Extensions.AspNetCore.Configuration.Secrets;
using Azure.Identity;
using InterventionManagement.Api.Infrastructure;
using InterventionManagement.Api.Web.Auth;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.Identity.Web;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Security.Cryptography;

var builder = WebApplication.CreateBuilder(args);
builder.Configuration.AddJsonFile("appsettings.local.json", optional: true, reloadOnChange: true);

var vaultUri = builder.Configuration["KeyVault:VaultUri"];
if (!string.IsNullOrEmpty(vaultUri))
{
    builder.Configuration.AddAzureKeyVault(new Uri(vaultUri), new DefaultAzureCredential(), new
    AzureKeyVaultConfigurationOptions());
}

var enableSso = builder.Configuration.GetValue("AuthenticationFeatures:EnableSSO", false);
var localJwtSigningKey = new SymmetricSecurityKey(RandomNumberGenerator.GetBytes(64));
builder.Services.AddSingleton(localJwtSigningKey);

```



```

builder.Services.AddScoped<LocalJwtTokenService>();

var authBuilder = builder.Services.AddAuthentication(options =>
{
    options.DefaultScheme = AuthenticationSchemes.DynamicBearer;
    options.DefaultChallengeScheme = AuthenticationSchemes.DynamicBearer;
});

authBuilder.AddPolicyScheme(AuthenticationSchemes.DynamicBearer, AuthenticationSchemes.DynamicBearer,
options =>
{
    options.ForwardDefaultSelector = context =>
    {
        if (!enableSso) return AuthenticationSchemes.LocalBearer;

        var authorization = context.Request.Headers.Authorization.ToString();
        if (!authorization.StartsWith("Bearer ", StringComparison.OrdinalIgnoreCase))
        {
            return AuthenticationSchemes.EntraBearer;
        }

        var token = authorization["Bearer ".Length..].Trim();
        var issuer = new JwtSecurityTokenHandler().CanReadToken(token)
            ? new JwtSecurityTokenHandler().ReadJwtToken(token).Issuer
            : string.Empty;

        return issuer.Equals(builder.Configuration["LocalJwt:Issuer"] ??
"InterventionManagement.Local", StringComparison.OrdinalIgnoreCase)
            ? AuthenticationSchemes.LocalBearer
            : AuthenticationSchemes.EntraBearer;
    };
});

authBuilder.AddJwtBearer(AuthenticationSchemes.LocalBearer, options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidIssuer = builder.Configuration["LocalJwt:Issuer"] ?? "InterventionManagement.Local",
        ValidateAudience = true,
        ValidAudience = builder.Configuration["LocalJwt:Audience"] ?? "InterventionManagement.Api",
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = localJwtSigningKey,
        ValidateLifetime = true,
        ClockSkew = TimeSpan.FromMinutes(2),
        RoleClaimType = ClaimTypes.Role
    };
});

if (enableSso)
{
    authBuilder.AddMicrosoftIdentityWebApi(
        builder.Configuration.GetSection("AzureAd"),
        jwtBearerScheme: AuthenticationSchemes.EntraBearer);
}

builder.Services.AddAuthorization();
builder.Services.AddScoped<IClaimsTransformation, DatabaseRoleClaimsTransformation>();
builder.Services.AddInfrastructure(enableSso);

if (enableSso)
{

```

```

builder.Services.AddMicrosoftGraph(options =>
{
    options.BaseUrl = builder.Configuration["Graph:BaseUrl"] ?? "https://graph.microsoft.com/
v1.0";
    options.Scopes = builder.Configuration["Graph:Scopes"] ?? "User.Read Mail.Send";
});
}

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Paste a local token from /api/auth/local-login, or an Entra ID access token."
    });

    options.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        [new OpenApiSecurityScheme { Reference = new OpenApiReference { Type =
ReferenceType.SecurityScheme, Id = "Bearer" } }] = []
    });

    if (enableSso)
    {
        var tenantId = builder.Configuration["AzureAd:TenantId"];
        options.AddSecurityDefinition("EntraID", new OpenApiSecurityScheme
        {
            Type = SecuritySchemeType.OAuth2,
            Flows = new OpenApiOAuthFlows
            {
                AuthorizationCode = new OpenApiOAuthFlow
                {
                    AuthorizationUrl = new Uri($"https://login.microsoftonline.com/{tenantId}/oauth2/
v2.0/authorize"),
                    TokenUrl = new Uri($"https://login.microsoftonline.com/{tenantId}/oauth2/v2.0/
token"),
                    Scopes =
builder.Configuration.GetSection("Swagger:OAuthScopes").Get<Dictionary<string, string>>() ?? []
                }
            }
        });
    }
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(options =>
    {
        options.OAuthClientId(builder.Configuration["Swagger:OAuthClientId"]);
        options.OAuthUsePkce();
    });
}

```

```

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();

```

Controllers/AuthController.cs

```

using InterventionManagement.Api.Web.Auth;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace InterventionManagement.Api.Web.Controllers;

[ApiController]
[Route("api/auth")]
[AllowAnonymous]
public sealed class AuthController(IConfiguration configuration, LocalJwtTokenService tokenService) :
    ControllerBase
{
    [HttpPost("local-login")]
    public ActionResult<LocalLoginResponse> LocalLogin(LocalLoginRequest request)
    {
        if (!configuration.GetValue("AuthenticationFeatures:EnableLocalLogin", false))
        {
            return NotFound();
        }

        if (string.IsNullOrEmpty(request.Username) ||
            string.IsNullOrEmpty(request.Password))
        {
            return Unauthorized();
        }

        var objectId = request.Username.Equals("approver", StringComparison.OrdinalIgnoreCase)
            ? "00000000-0000-0000-0000-000000000111"
            : "00000000-0000-0000-0000-000000000222";

        return Ok(new LocalLoginResponse(tokenService.CreateToken(request.Username, objectId),
            "Bearer", 3600));
    }
}

public sealed record LocalLoginRequest(string Username, string Password);
public sealed record LocalLoginResponse(string AccessToken, string TokenType, int ExpiresIn);

```

Web/Auth/DatabaseRoleClaimsTransformation.cs

```

using System.Security.Claims;
using InterventionManagement.Api.Application.Abstractions;
using Microsoft.AspNetCore.Authentication;

namespace InterventionManagement.Api.Web.Auth;

public sealed class DatabaseRoleClaimsTransformation(IUserRoleRepository roles) :
    IClaimsTransformation
{
    {

```

```

private const string TransformationMarker = "imp.roles.transformed";

public async Task<ClaimsPrincipal> TransformAsync(ClaimsPrincipal principal)
{
    if (principal.HasClaim(TransformationMarker, "true"))
    {
        return principal;
    }

    var objectId = principal.FindFirstValue("oid")
        ?? principal.FindFirstValue("http://schemas.microsoft.com/identity/claims/objectidentifier");

    if (string.IsNullOrEmpty(objectId) || principal.Identity is not ClaimsIdentity identity)
    {
        return principal;
    }

    foreach (var tokenRole in identity.Claims
        .Where(claim => claim.Type == ClaimTypes.Role || claim.Type == identity.RoleClaimType ||
        claim.Type == "roles")
        .ToArray())
    {
        identity.RemoveClaim(tokenRole);
    }

    foreach (var role in await roles.GetRolesForObjectAsync(objectId, CancellationToken.None))
    {
        identity.AddClaim(new Claim(ClaimTypes.Role, role));
    }

    identity.AddClaim(new Claim(TransformationMarker, "true"));
    return principal;
}
}

```

Infrastructure/Email/EmailServices.cs

```

using InterventionManagement.Api.Application.Abstractions;
using Microsoft.Graph;
using Microsoft.Extensions.Logging;

namespace InterventionManagement.Api.Infrastructure.Email;

public sealed class GraphEmailService(GraphServiceClient graphClient) : IEmailService
{
    public async Task SendInterventionNotificationAsync(string mailbox, string recipient, string
subject, string body, CancellationToken cancellationToken)
    {
        var message = new Message
        {
            Subject = subject,
            Body = new ItemBody { ContentType = BodyType.Text, Content = body },
            From = new Recipient { EmailAddress = new EmailAddress { Address = mailbox } },
            ToRecipients = [ new Recipient { EmailAddress = new EmailAddress { Address =
recipient } } ]
        };

        await graphClient.Me.SendMail(message, true).Request().PostAsync(cancellationToken);
    }
}

```

```
    }  
}  
  
public sealed class LocalEmailService(ILogger<LocalEmailService> logger) : IEmailService  
{  
    public Task SendInterventionNotificationAsync(string mailbox, string recipient, string subject,  
string body, CancellationToken cancellationToken)  
    {  
        logger.LogInformation("Local email notification suppressed. Mailbox: {Mailbox}, Recipient:  
{Recipient}, Subject: {Subject}", mailbox, recipient, subject);  
        return Task.CompletedTask;  
    }  
}
```